Multi-Robot Warehouse Automation With Priority-Based Scheduling Using Ros2 and Gazebo

Project Overview:

In this project, we create a warehouse automation system, where various robots collaborate to accomplish different tasks related to transporting packages. The system consists of two mobile robots, one heavy payload AMR, one small payload AMR, and one quadcopter drone, all developed through ROS 2 Jazzy and Gazebo Harmonic. Tasks are prioritized and completed by the robots according to their priority, while a collision avoidance system controls robots movements for ensuring safety and efficiency. Users can schedule tasks, place orders, and keep an eye on robots and packages in real time via an interactive dashboard linked to Gazebo.

1. Files Submitted

File	Purpose
warehouse.world	Gazebo simulation world containing the warehouse layout, including walls, shelves, pillars, packages, and the initial positions of all robots.
warehouse_amr.urdf	URDF model of the first autonomous mobile robot (AMR) equipped with differential drive, LiDAR, four wheels, and two caster wheels.
warehouse_amr2.urdf	URDF model of the second small AMR. It has the same mechanical design as the first AMR but operates using separate ROS topics.
warehouse_big_amr.urdf	URDF model of the heavy-load AMR responsible for transporting large packages within the warehouse.
warehouse_drone.urdf	URDF model of the quadcopter drone used for aerial package transportation and monitoring tasks.
drone_position_controller.py	Python script implementing the PID-based position controller for autonomous drone navigation.
warehouse_master.py	Main backend: priority scheduler, all 4 robot controllers, collision avoidance, WebSocket server.
warehouse_ui.html	Web-based dashboard that enables users to monitor robot activities, submit warehouse orders, and interact with the simulation in real time.
bridge_config.yaml	Configuration file used by the ROS2–Gazebo bridge to establish communication between simulation and ROS topics, such as velocity commands, odometry, and LiDAR data.
package.xml / CMakeLists.txt	ROS2 package configuration files required for building the project using the colcon build system.

2. Prerequisites

- Ubuntu with ROS2 Jazzy installed
- Gazebo Harmonic (gz sim) installed and on PATH
- ros_gz_bridge package installed
- Python 3 with pip

Install the one Python dependency not part of ROS2 (one-time):

```
pip install websockets --break-system-packages
```

3. Run Order

Three terminals, started in this exact order, and then a browser tab:

Terminal	Runs
T1	Launch the Gazebo simulation.
T2	Start the ROS2–Gazebo bridge
T3	Launch warehouse_master.py
Browser	Open warehouse_ui.html (dashboard) in a browser

4. Terminal Commands

Terminal 1 — Launch Gazebo

```
source /opt/ros/jazzy/setup.bash
gz sim ~/mrwa_ws/src/warehouse_world/worlds/warehouse.world -r
```

Wait until the warehouse, shelves, and all four robots (drone + 2 small AMRs + big AMR) are visible and spawned before continuing.

Terminal 2 — ROS2 / Gazebo Bridge

```
source /opt/ros/jazzy/setup.bash
ros2 run ros_gz_bridge parameter_bridge \
  "/world/multi_robot_warehouse/set_pose@ros_gz_interfaces/srv/SetEntityPose" \
  --ros-args -p config_file:=~/mrwa_ws/src/warehouse_world/config/bridge_config.yaml
```

To confirm that the bridge has started successfully, run the following commands in another terminal:

```
ros2 topic list | grep cmd_vel
ros2 service list | grep set_pose
```

The topic list should show cmd_vel topics for the drone and all three AMRs; the service list should show /world/multi_robot_warehouse/set_pose.

Terminal 3 — Warehouse Controller

```
source /opt/ros/jazzy/setup.bash
cd ~/mrwa_ws/src/warehouse_world/scripts
pip install websockets --break-system-packages   # one-time only
python3 warehouse_master.py
```

On success it prints something like:

```
UI server: ws://localhost:8765 — open warehouse_ui.html
```

Browser — Open the Dashboard

Open warehouse_ui.html directly in a browser :

```
chrome ~/mrwa_ws/warehouse_ui.html
```

The dashboard should show "connected" at the top and four robot cards (Drone, AMR, AMR 2, Big AMR).

5. Usage

- Use the PLACE ORDER panel to select a package, choose its priority (Urgent, Normal, or Low), and click ADD ORDER to submit the task.
- To quickly demonstrate the system, click LOAD DEMO QUEUE. This automatically adds a predefined set of orders for all four robots..
- The Activity Log displays the current status of each robot, including task progress and collision avoidance messages such as WAIT, GIVING WAY, and UNSTICK.

6. Troubleshooting Quick Reference

Issue	Solution/Fix
Gazebo does not load the warehouse	Check that the `<uri>` paths in `warehouse.world` point to the correct location on your system.
Robots are not moving	Make sure the project is started in the following order: Gazebo, ROS2 Bridge, and then `warehouse_master.py`. If the robots still do not move, restart the application and try again.
“ERROR: pip install websockets…” on launch	Run: pip install websockets --break-system-packages
Dashboard shows "Disconnected"	Ensure that `warehouse_master.py` is running before opening `warehouse_ui.html`. If the backend is already running, simply refresh the browser.

7. Future Improvements

- Replace the current pose-based movement with physics-based navigation using `cmd_vel` for more realistic robot motion.
- Implement a physical pick-and-place mechanism instead of simulated package teleportation.
- Generate robot paths dynamically from the warehouse map, eliminating the need for manually defined routes.

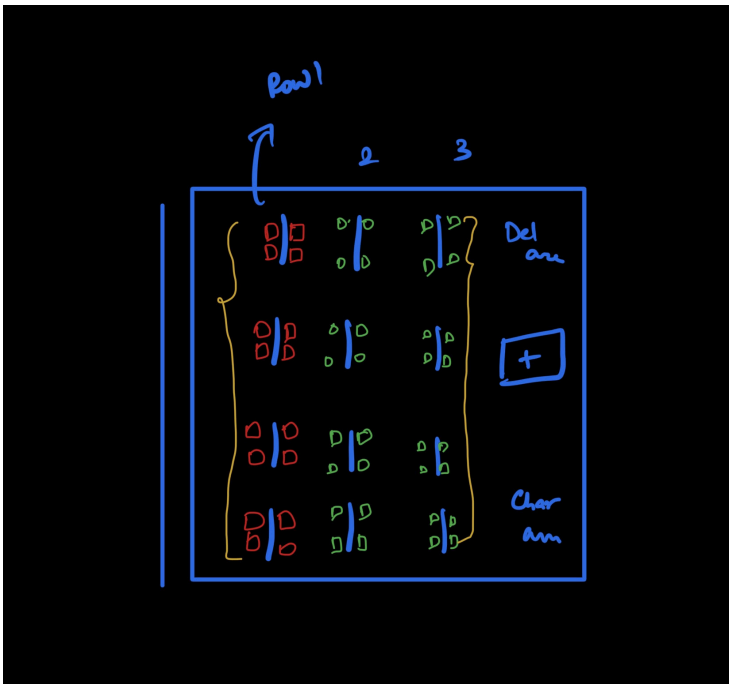
8. Use of AI Tools

AI was used to generate the initial Gazebo warehouse environment based on a pseudocode and a user-provided image description. The generated environment served as a starting template for further development and customization.

AI Tool Used: Claude

Prompt Used:
"Based on the provided hand-drawn warehouse layout, generate a basic Gazebo world file containing the initial structure, including walls, floor, and approximate placement of warehouse elements."

All final design decisions, implementation, modifications, and validation were performed by the project team.



Reference : Hand- drawn warehouse layout(Initial idea)

9. Contribution to the project

This project's system architecture is divided into 4 layers and the team is divided into two technical sides.

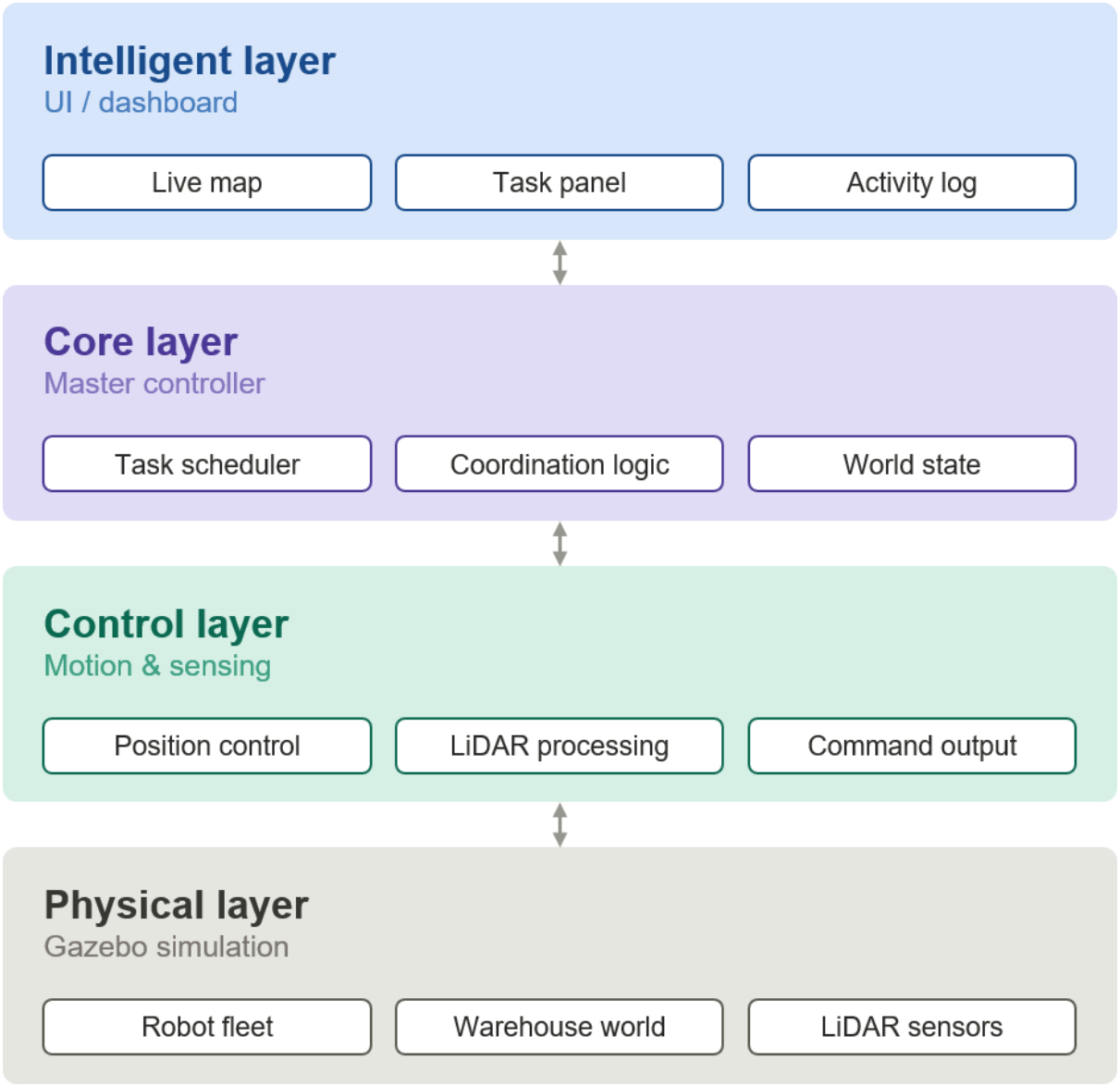
1.Simulation Environment and Robot Modeling , Drone Flight Controller (PID) , AMR Navigation ,Task Scheduling and System Integration.

- Sruthika Maheswaran Kalaivani
- Mohanram Swaminathan
- Ronal Roy

2.Multi - robot collision avoidance , Dashboard UI , Testing and Documentation

- Shruthi Suresh

- Dhanush Palani Vijay Kumari



Reference: System Architecture for Multi-robot warehouse automation